Centralisation de l'État Global avec Redux Toolkit

Objectifs

1. Installer et configurer Redux Toolkit (`@reduxjs/toolkit`) et le pont React (`react-redux`) dans une application existante.
2. Concevoir une tranche d'état métier (*Slice*) regroupant l'état initial, les reducers immuables et les créateurs d'actions.
3. Assembler le Store global de l'application et l'injecter au sommet de l'arborescence React via le composant `<Provider>`.
4. Remplacer le passage de `props` en profondeur (*Prop Drilling*) par une lecture directe de l'état avec `useSelector`.
5. Déclencher des mutations d'état prévisibles depuis n'importe quel composant avec `useDispatch`.
6. Inspecter le flux d'actions et utiliser le débogage temporel avec l'extension Redux DevTools.

Contexte du Projet : OmniMonitor - État Partagé Multi-Vues

Au fur et à mesure que l'application **OmniMonitor** grandit, plusieurs composants éloignés (le bandeau d'en-tête, la grille des stations, le panneau de statistiques et le module d'alertes) ont besoin d'accéder simultanément à la même liste de stations météo et au même filtre actif.

Votre mission consiste à migrer la gestion des stations depuis l'état local (`useState` dans `App.jsx`) vers un Store global **Redux Toolkit**, afin que chaque composant puisse lire ou mettre à jour les données de manière autonome.

Étape 1 : Installation des Dépendances

1. Ouvrez un terminal à la racine de votre projet `omnimonitor-app`.
2. Installez les deux paquets officiels requis pour utiliser Redux moderne avec React :

```shell
npm install @reduxjs/toolkit react-redux
```

3. Vérifiez dans votre fichier `package.json` que `@reduxjs/toolkit` et `react-redux` apparaissent bien dans la section `dependencies`.

Étape 2 : Création de la Tranche d'État (`stationsSlice.js`)

Vous allez isoler toute la logique métier liée aux stations d'observation dans un fichier dédié.

1. Créez un nouveau dossier `src/store/`.
2. Créez le fichier `src/store/stationsSlice.js`.
3. Définissez un état initial (`initialState`) contenant :
    - `list` : un tableau contenant au moins 3 stations météo (avec `id`, `name`, `altitude`, `status`, `sensorsCount`).
    - `filterStatus` : une chaîne de caractères initialisée à `'all'` (valeurs possibles : `'all'`, `'online'`, `'offline'`).
    - `searchQuery` : une chaîne de caractères initialisée à `''` pour la recherche textuelle.
4. Utilisez `createSlice` pour créer la tranche nommée `'stations'` avec les 4 fonctions reducers suivantes :
    - `addStation(state, action)` : ajoute la nouvelle station reçue dans `action.payload` à la fin de `state.list`.
    - `removeStation(state, action)` : supprime la station dont l'identifiant correspond à `action.payload`.
    - `toggleStationStatus(state, action)` : inverse le statut (`'online'` / `'offline'`) de la station ciblée par `action.payload`.
    - `setFilterStatus(state, action)` : remplace la valeur de `state.filterStatus` par `action.payload`.
5. Exportez individuellement les créateurs d'actions générés (`stationsSlice.actions`) ainsi que le reducer par défaut (`stationsSlice.reducer`).

JavaScript

```
// Structure attendue dans src/store/stationsSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
```

```
  list: [
    { id: 1, name: 'Toulouse-Blagnac', altitude: 152, status: 'online',
sensorsCount: 6 },
    { id: 2, name: 'Pic du Midi', altitude: 2877, status: 'offline', sensorsCount:
3 },
    { id: 3, name: 'Marseille-Marignane', altitude: 22, status: 'online',
sensorsCount: 5 },
    { id: 4, name: 'Brest-Guipavas', altitude: 99, status: 'online', sensorsCount:
4 }
  ],
  filterStatus: 'all'
};

const stationsSlice = createSlice({
  name: 'stations',
  initialState,
  reducers: {
    addStation: (state, action) => {
      state.list.push(action.payload);
    },
    removeStation: (state, action) => {
      state.list = state.list.filter(st => st.id !== action.payload);
    },
    toggleStationStatus: (state, action) => {
      const target = state.list.find(st => st.id === action.payload);
      if (target) {
        target.status = target.status === 'online' ? 'offline' : 'online';
      }
    },
    setFilterStatus: (state, action) => {
      state.filterStatus = action.payload;
    }
  }
});

export const {
  addStation,
  removeStation,
  toggleStationStatus,
  setFilterStatus
} = stationsSlice.actions;

export default stationsSlice.reducer;
```

Étape 3 : Configuration du Store Global et Injection du `<Provider>`

1. Créez le fichier `src/store/store.js`.
2. Importez `configureStore` depuis `'@reduxjs/toolkit'` et votre `stationsReducer` depuis `'./stationsSlice'`.
3. Créez et exportez la constante `store` en associant la clé `stations` à votre reducer :

```javascript
// src/store/store.js
import { configureStore } from '@reduxjs/toolkit';
import stationsReducer from './stationsSlice';

export const store = configureStore({
  reducer: {
    stations: stationsReducer
  }
});
```

4. Ouvrez le point d'entrée de l'application `src/main.jsx`.
5. Importez le composant `Provider` depuis `'react-redux'` ainsi que votre `store` depuis `'./store/store.js'`.
6. Enveloppez le composant `<App />` à l'intérieur de `<Provider store={store}>`.

Étape 4 : Lecture de l'État Global dans l'En-Tête (`Header.jsx`)

Votre composant `Header` n'a plus besoin de recevoir de `props` depuis `App` pour afficher les compteurs globaux.

1. Ouvrez `src/components/Header.jsx`.
2. Importez le Hook `useSelector` depuis `'react-redux'`.
3. Extrayez la liste des stations depuis le Store :

```javascript
const stations = useSelector(state => state.stations.list);
```

4. Calculez dynamiquement le nombre total de stations et le nombre de stations en ligne (`status === 'online'`).
5. Affichez ces compteurs en temps réel dans la bannière d'en-tête.

Étape 5 : Expédition d'Actions avec `useDispatch`

Vous allez maintenant créer une barre de filtrage et connecter les cartes de stations directement au Store Redux.

1. Créez un composant `src/components/FilterBar.jsx` :
    - Utilisez `useSelector` pour lire le filtre actif (`state.stations.filterStatus`).
    - Utilisez `useDispatch` pour récupérer la fonction `dispatch`.
    - Affichez trois boutons (`Toutes`, `En ligne`, `Hors ligne`) qui appellent au clic `dispatch(setFilterStatus('all'))`, `dispatch(setFilterStatus('online'))` ou `dispatch(setFilterStatus('offline'))`.
    - Ajoutez un bouton `"Ajouter une station de test"` qui expédie l'action `addStation` avec un nouvel objet station généré dynamiquement (`id: Date.now()`).
2. Mettez à jour `src/components/StationCard.jsx` :
    - Importez `useDispatch` et les actions `toggleStationStatus` et `removeStation`.
    - Ajoutez un bouton permettant de basculer le statut (`online` / `offline`) via `dispatch(toggleStationStatus(id))`.
    - Modifiez le bouton de suppression pour qu'il appelle directement `dispatch(removeStation(id))` sans passer par une prop `onDelete` du parent.
3. Mettez à jour `src/App.jsx` :
    - Supprimez les anciens `useState` devenus inutiles.
    - Lisez `list` et `filterStatus` via `useSelector(state => state.stations)`.

- Filtrez la liste des stations selon `filterStatus` avant d'afficher les composants `<StationCard />`.

Étape 6 : Débogage et Voyage Temporel avec Redux DevTools

1. Ouvrez votre application dans le navigateur et ouvrez les outils de développement (`F12`).
2. Sélectionnez l'onglet **Redux** (extension Redux DevTools).
3. Effectuez plusieurs actions dans l'interface : changez le filtre, basculez le statut de deux stations, ajoutez une station puis supprimez-en une.
4. Observez la colonne de gauche dans Redux DevTools :
    - Vérifiez que chaque action apparaît avec son préfixe (`stations/toggleStationStatus`, `stations/setFilterStatus`, etc.).
    - Cliquez sur une action et inspectez les sous-onglets **Action** (vérifiez le contenu de `payload`), **State** (arbre complet) et **Diff** (propriétés modifiées uniquement).
5. Utilisez le curseur temporel en bas de la fenêtre Redux DevTools pour revenir en arrière et rejouer la séquence d'actions en direct sous vos yeux.

Critères de Validation

Avant de valider ce segment, vérifiez que :

- Le composant `<Provider store={store}>` enveloppe bien l'application dans `src/main.jsx`.
- Le composant `App.jsx` ne passe plus de fonctions callbacks (`onDelete`, `onToggle`) en cascade à ses enfants.
- La modification du statut d'une station depuis sa carte met instantanément à jour le compteur situé dans le composant `Header`.
- Toutes les mutations écrites dans `stationsSlice.js` fonctionnent sans erreur et sont visibles dans Redux DevTools.